

Software Requirements Specification

for

BuildFlow

Smart Custom PC Builder & Intelligent Order Fulfillment System

Prepared in accordance with the IEEE SRS Standard (IEEE 830 / 29148)

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete and detailed description of the requirements for BuildFlow, a Smart Custom PC Builder and Intelligent Order Fulfillment System. This document describes the system's functionality, interfaces, constraints, and performance requirements. It is intended to be used by developers, project managers, testers, warehouse and assembly staff, quality assurance engineers, logistics personnel, and other stakeholders to understand the system requirements clearly.

1.2 Document Conventions

This document follows the IEEE Software Requirements Specification (SRS) standard.

- Shall indicates a mandatory requirement.
- Should indicates a recommended requirement.
- May indicates an optional feature.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Customers who will configure and purchase custom PCs
- Warehouse managers responsible for inventory
- Assembly technicians building customer orders
- QA engineers responsible for quality testing
- Logistics managers handling packaging and shipment
- System administrators
- Software developers and testers
- Project managers and stakeholders

Readers new to the system should begin with Section 1 and Section 2 for an overview, while technical users may focus on Sections 3 and 4.

1.4 Product Scope

BuildFlow is a web-based platform, developed using the MERN stack, that integrates the complete lifecycle of a custom PC order into a single system. It allows customers to configure compatible custom PC builds and complete purchases, while also managing the enterprise-level operational workflow that follows — including inventory reservation, component allocation, assembly management, quality assurance, packaging, shipment tracking, and analytics.

Unlike traditional custom PC builder websites that stop functioning once payment is completed, BuildFlow connects customers, warehouse personnel, assembly technicians, QA engineers, logistics staff, and administrators through one centralized system with role-based dashboards, transparent order tracking, and permanent audit logging. The system aims to eliminate inventory conflicts, prevent duplicate operations, and provide real-time visibility into the entire order fulfillment process.

1.5 References

- IEEE Software Requirements Specification Standard

- BuildFlow Project Proposal Document (Problem Statement & System Overview)
 - MERN Stack (MongoDB, Express.js, React.js, Node.js) documentation
-

2. Overall Description

2.1 Product Perspective

BuildFlow is a standalone web-based application that integrates customer-facing e-commerce functionality with enterprise-level order fulfillment operations. It may integrate with third-party services such as payment gateways, shipment/courier APIs, and email/SMS notification services. It follows a client-server architecture built on the MERN stack (MongoDB, Express.js, React.js, Node.js) and supports access through web browsers, with responsive design for desktop and mobile devices.

2.2 Product Functions

Customer Module

- Registration/Login: Allows customers to create accounts and log in securely.
- Component Selection: Allows customers to select hardware components (CPU, GPU, Motherboard, RAM, SSD/HDD, PSU, Cabinet, Cooler) for a custom build.
- Compatibility Validation: Automatically checks compatibility across selected components.
- Save Build: Allows customers to save an in-progress or completed build.
- Build Comparison: Allows customers to compare two or more saved builds.
- Shopping Cart: Allows customers to add builds/components and manage quantities.
- Order Placement: Allows customers to check out and pay for an order.
- Order Tracking: Allows customers to view real-time order status and history.

Warehouse Module

- Inventory Management: Allows warehouse staff to view and adjust stock.
- Stock Reservation: Automatically reserves inventory once payment is verified.
- Component Allocation: Deducts and links reserved stock to specific orders.
- Low Stock Alerts: Notifies staff when stock falls below a threshold.

Assembly Module

- Assembly Queue: Lists orders awaiting assembly.
- Technician Assignment: Assigns orders to assembly technicians.
- Build Status / Progress Updates: Tracks assembly progress and completion time.

Quality Assurance Module

- Testing Reports: Allows QA engineers to record test results.
- Quality Verification: Confirms the build meets quality standards.
- Pass/Fail Decision: Routes the order to packaging or back to assembly.

Logistics Module

- Packaging: Records packaging completion.

- Shipment Creation: Generates a shipment and tracking number.
- Delivery Tracking: Tracks shipment status through to delivery.
- Failed Shipment Handling: Flags and manages failed delivery attempts.

Administration Module

- User Management: Creates, updates, and deactivates user accounts across all roles.
- Component Management: Manages the hardware component catalog.
- Analytics: Displays business and operational insights.
- Reports: Generates and exports operational reports.
- System Monitoring: Tracks queues and failed operations.
- Audit Logs: Provides a permanent, searchable record of all critical operations.

2.3 User Classes and Characteristics

2.3.1 Customer

- Configures custom PC builds
- Places and tracks orders
- Requires no technical knowledge of hardware compatibility

2.3.2 Warehouse Manager

- Manages inventory levels
- Reserves and allocates components
- Requires operational/warehouse domain knowledge

2.3.3 Assembly Technician

- Assembles ordered systems
- Updates build progress
- Requires hardware assembly expertise

2.3.4 QA Engineer

- Tests completed builds
- Approves or rejects build quality
- Requires technical testing knowledge

2.3.5 Logistics Manager

- Packages orders
- Dispatches and tracks shipments
- Requires logistics/courier coordination knowledge

2.3.6 Administrator

- Manages users, catalog, and system configuration
- Monitors analytics and system health
- Requires technical/administrative knowledge

2.4 Operating Environment

- Operating Systems: Windows, macOS, Linux (for staff/admin devices); Android, iOS (for customer mobile browsing)
- Web Browsers: Chrome, Firefox, Edge, Safari
- Server Environment: Cloud or on-premise hosting supporting Node.js/Express.js backend and MongoDB database

2.5 Design and Implementation Constraints

- The system shall be developed using the MERN stack (MongoDB, Express.js, React.js, Node.js).
- The system shall comply with web security standards, including JWT-based authentication and password hashing.
- The platform shall support a scalable architecture to accommodate a growing user base and component catalog.
- The system shall ensure transactional consistency for inventory operations to prevent overselling.
- The system shall maintain an append-only audit log; no operational record shall be deleted.

2.6 User Documentation

- User manuals for customers, warehouse staff, assembly technicians, QA engineers, and logistics staff
- Administrator guide
- Online help and FAQs
- API documentation for developers

2.7 Assumptions and Dependencies

- Users have access to the internet and a modern web browser
- Users possess basic computer literacy
- The system depends on the availability of third-party payment gateway and shipment/courier APIs
- System availability depends on hosting infrastructure

3. External Interface Requirements

3.1 User Interfaces

The system shall provide the following interfaces:

- Login and Registration Screen
- Role-based Dashboards (Customer, Warehouse Manager, Assembly Technician, QA Engineer, Logistics Manager, Administrator)
- Component Catalog and Custom PC Builder page with live compatibility feedback
- Shopping Cart and Checkout interface
- Order Tracking and Order History page
- Assembly Queue and Progress Update interface
- QA Testing Report interface

- Packaging and Shipment Management interface
- Analytics and Reporting Dashboard
- Audit Log Viewer
- Profile and Settings page

3.2 Hardware Interfaces

- Server hardware supporting scalable load and concurrent transaction processing
- User devices such as desktops, laptops, tablets, and smartphones

3.3 Software Interfaces

- Database Management System: MongoDB with Mongoose ODM
- Payment Gateway API for order payment processing
- Shipment/Courier API for shipment generation and delivery tracking
- Email and SMS notification services
- Cloud image storage service (optional, e.g., Cloudinary) for component images
- Socket.IO for real-time dashboard and status updates

3.4 Communications Interfaces

- HTTPS for secure communication
- REST APIs for client-server and third-party integration
- WebSocket (Socket.IO) for real-time updates
- Email and push/SMS notification protocols

4. System Features

4.1 Custom PC Builder & Compatibility Validation

BuildFlow allows customers to visually select hardware components and assemble a custom PC build. Components are organized by category, and the system validates compatibility in real time as the customer builds.

Customers can:

- Select one component per required category (CPU, GPU, Motherboard, RAM, SSD/HDD, PSU, Cabinet, Cooler)
- View live compatibility warnings for socket type, chipset, RAM type, PSU wattage, storage interface, cabinet size, and cooler support
- View the running total price as components are added or changed
- Save, edit, delete, compare, and share builds

4.2 Intelligent Inventory Reservation

Once payment succeeds, BuildFlow automatically reserves the inventory required for the order using atomic transactions. This prevents stock conflicts, double allocation, and overselling across simultaneous orders. Reserved inventory is released automatically if the order is cancelled or payment fails.

4.3 Order Fulfillment Workflow Engine

Every order progresses through a predefined state machine, giving all roles a shared, real-time view of order status:

- Order Created
- Payment Verified
- Inventory Reserved
- Assembly Assigned
- Assembly Completed
- QC Testing
- QC Passed / QC Failed
- Packaging
- Shipping
- Delivered

The system enforces valid state transitions only, preventing orders from skipping stages except through explicitly defined exception flows (e.g., QC Failed routes back to Assembly).

4.4 Assembly & Quality Assurance Management

Assembly technicians view a queue of orders awaiting assembly, update build progress, and mark builds complete, which automatically routes the order to the QA queue. QA engineers record testing results and approve or reject the build; a failed build is routed back to assembly with failure notes, while a passed build proceeds to packaging.

4.5 Packaging, Shipment & Delivery Tracking

Logistics managers package QC-approved orders and generate shipments through the shipment API, producing a tracking number that is shared with the customer. The system tracks delivery status through to confirmed delivery and provides a defined process for handling failed shipment attempts, including rescheduling or return-to-warehouse actions.

4.6 Duplicate Action Prevention

Critical operations — including payment confirmation, inventory reservation, shipment generation, and assembly completion — execute exactly once per order. The system uses idempotency checks and transaction locks to reject duplicate requests caused by network retries, page refreshes, or concurrent staff actions.

4.7 Permanent Audit Logging

Every critical operation is recorded in an immutable, append-only audit log, including the responsible actor, timestamp, action type, and before/after state. No audit records are ever deleted or modified, providing a complete and trustworthy operational history for every order.

4.8 Automated Retry Mechanism

If an external operation fails — such as a shipment API call, email, SMS, or payment confirmation — the system automatically retries the operation a configurable number of times with backoff before marking the process as failed and flagging it for administrator attention.

4.9 Dashboard & Analytics

Role-based dashboards provide real-time visibility into operations. The Administrator analytics dashboard provides insights into:

- Daily orders and revenue
- Component sales
- Inventory status
- Build completion rate
- Delivery performance
- Warehouse efficiency
- Assembly time
- Technician productivity

Dashboards update in real time via Socket.IO as underlying order and inventory data changes.

5. Functional Requirements

FR-1: User Management & Access Control

FR-1.1 Registration

- The system shall allow users to register with name, email, password, contact number, and role (default: Customer).
- The system shall validate input, check for duplicate email, hash the password (bcrypt), and create the user record.
- The system shall create the account and send a confirmation email/message on success.

FR-1.2 Login

- The system shall authenticate users via email and password.
- The system shall generate a JWT access token and refresh token on successful login and redirect the user to their role-based dashboard.

FR-1.3 Password Reset

- The system shall generate a time-limited reset link/OTP for a registered email and send it to the user.
- The system shall update the password only after the reset link/OTP is verified.

FR-1.4 Role-Based Access Control

- The system shall check the role embedded in the JWT token before granting access to any endpoint or dashboard.
- The system shall grant or deny access based on role: Customer, Warehouse Manager, Assembly Technician, QA Engineer, Logistics Manager, or Administrator.

FR-1.5 Profile Management

- The system shall allow users to update their name, address, contact number, and avatar, and shall validate and save the changes.

FR-1.6 User Administration

- The system shall allow only the Administrator to create, update, deactivate, delete, or change the role of a user account.
- The system shall log every user administration change to the audit trail.

FR-2: Component Catalog & Custom PC Builder

FR-2.1 Add/Edit/Delete Component

- The system shall allow the Administrator to create, update, and delete component records, including specifications, price, stock, and images, and reflect changes in the catalog instantly.

FR-2.2 Browse & Search Components

- The system shall allow customers to search and filter components by keyword, category, price range, and brand, and shall return matching components with specs, price, and stock status.

FR-2.3 Component Selection & Build Creation

- The system shall allow customers to select one component per required category (CPU, GPU, Motherboard, RAM, SSD/HDD, PSU, Cabinet, Cooler) and shall display a live build summary with running total price.

FR-2.4 Compatibility Validation Engine

- The system shall automatically validate CPU socket, motherboard chipset, RAM type, PSU wattage, storage interface, cabinet size, and cooler support whenever a component is added or changed.
- The system shall display compatibility conflicts, if any, and shall block checkout until all conflicts are resolved.

FR-2.5 Save/Edit/Delete Build

- The system shall allow customers to save, edit, or delete a build under their account, and shall display saved builds under "My Builds."

FR-2.6 Compare & Share Builds

- The system shall allow customers to compare two or more saved builds in a side-by-side view.
- The system shall allow customers to generate a unique shareable link for a build.

FR-3: Order Placement & Payment

FR-3.1 Cart Management

- The system shall allow customers to add, remove, or update the quantity of builds/components in the cart and shall re-check current stock and compatibility on every change.

FR-3.2 Checkout

- The system shall capture shipping address and delivery preference, re-validate stock/compatibility, and calculate the final order total (including tax/shipping) before payment.

FR-3.3 Payment Processing

- The system shall process payment through an integrated payment gateway and shall apply an idempotency check to prevent duplicate charges.
- The system shall generate a unique Order ID upon successful payment.

FR-3.4 Order Tracking & History

- The system shall display real-time order status on a tracking page and shall maintain a complete, permanent order history for each customer.
- The system shall allow customers to download an invoice and reorder from a past order.

FR-4: Inventory & Warehouse Management

FR-4.1 Inventory Reservation

- The system shall automatically reserve inventory for all order components immediately after payment verification, using atomic transactions to prevent overselling.

FR-4.2 Component Allocation

- The system shall deduct reserved stock from available inventory and link components to the order once it is ready for assembly.

FR-4.3 Manual Stock Adjustment

- The system shall allow the Warehouse Manager to adjust stock levels via an atomic transaction and shall log every inventory change.

FR-4.4 Low Stock Alerts

- The system shall continuously monitor stock levels and automatically alert the Warehouse Manager/Admin when stock falls below a configured threshold.

FR-4.5 Release Reserved Stock

- The system shall automatically release reserved inventory and restore stock if an order is cancelled or payment fails.

FR-5: Order Fulfillment Workflow (Assembly, QA & Logistics)

FR-5.1 State-Based Order Workflow

- The system shall move every order through a defined state machine: Created, Payment Verified, Inventory Reserved, Assembly Assigned, Assembly Completed, QC Testing, QC Passed/Failed, Packaging, Shipped, Delivered, and shall persist and expose the current state to relevant dashboards.

FR-5.2 Assembly Queue & Assignment

- The system shall list all orders with status "Inventory Reserved" in an assembly queue and shall allow an order to be assigned to an Assembly Technician.

FR-5.3 Assembly Progress & Completion

- The system shall allow the Assembly Technician to record timestamped progress updates and shall, on completion, calculate assembly duration, transition the order to "Assembly Completed," and notify QA.

FR-5.4 Quality Testing & Approval

- The system shall allow the QA Engineer to record a testing report and mark the order as QC Passed or QC Failed.
- The system shall route a QC Failed order back to Assembly with failure notes and shall not allow it to proceed to packaging.

FR-5.5 Packaging & Shipment Generation

- The system shall record packaging confirmation for QC Passed orders and shall generate a shipment with a tracking number exactly once per order.

FR-5.6 Delivery Tracking & Failed Shipment Handling

- The system shall track delivery status through to confirmed delivery and shall flag failed delivery attempts, offering reschedule or return-to-warehouse options.

FR-5.7 Duplicate Action Prevention

- The system shall use an idempotency key/transaction lock to ensure critical operations (payment confirmation, inventory reservation, shipment generation, assembly completion) execute exactly once.

FR-5.8 Admin Order Override

- The system shall allow the Administrator to manually override an order's state with a mandatory reason, which shall be permanently logged to the audit trail.

FR-6: Audit Logging, Notifications & Analytics

FR-6.1 Audit Logging

- The system shall write an immutable, append-only audit log entry for every critical operation, including actor, timestamp, action type, and before/after state, and shall not permit modification or deletion of existing entries.

FR-6.2 Audit Log Search

- The system shall allow the Administrator to search and filter audit logs by Order ID, actor, action type, and date range.

FR-6.3 Automated Retry Mechanism

- The system shall automatically retry failed external calls (shipment API, email, SMS, payment confirmation) up to a configured limit before marking the operation as failed and flagging it to the Administrator.

FR-6.4 Notifications

- The system shall send an email/SMS/in-app notification to the customer for each major order state change and shall log every notification attempt.

FR-6.5 Analytics Dashboard

- The system shall aggregate and display data on orders, revenue, top-selling components, inventory status, build completion rate, average assembly/delivery time, and technician productivity.

FR-6.6 Report Export

- The system shall allow the Administrator to export orders, inventory, or sales data as a downloadable CSV/PDF report.

FR-6.7 Real-Time Dashboard Updates

- The system shall push order/inventory state changes to subscribed dashboards in real time via Socket.IO, without requiring a page refresh.
-

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- The system shall support multiple concurrent users placing orders without data conflicts.
- Page and API response time shall remain below 3 seconds for normal operations.
- The system shall support increasing users and component catalog size without major architectural redesign.

6.2 Safety Requirements

- The system shall perform regular data backups.
- The system shall include fault tolerance and recovery mechanisms for failed external operations (payment, shipment, notifications).
- The system shall maintain transactional consistency for inventory and order state changes.

6.3 Security Requirements

- The system shall enforce secure authentication using JWT and password hashing (bcrypt).
- The system shall enforce role-based authorization on every endpoint.
- The system shall validate all user input to prevent injection and other common web attacks.
- The system shall use HTTPS for all client-server communication.

6.4 Software Quality Attributes

- **Usability:** Responsive interface with intuitive, role-specific dashboards.
- **Reliability:** Duplicate-action prevention and automated retry mechanisms ensure consistent operation outcomes.
- **Scalability:** Architecture supports growth in users, orders, and catalog size.
- **Maintainability:** Modular MERN-based architecture with clearly separated modules (Customer, Warehouse, Assembly, QA, Logistics, Admin).
- **Auditability:** Complete, permanent, append-only history of all critical operations.

Glossary

- **MERN Stack:** MongoDB, Express.js, React.js, and Node.js — the technology stack used to build BuildFlow.
- **Build:** A customer-configured set of compatible hardware components forming a custom PC.
- **Compatibility Validation Engine:** The subsystem that checks hardware compatibility across selected components.
- **Inventory Reservation:** The process of locking stock for an order immediately after payment to prevent overselling.
- **QC:** Quality Control — the testing and approval process performed before packaging.
- **Idempotency:** A property ensuring that a critical operation executes exactly once, even if requested multiple times.
- **Audit Log:** A permanent, append-only record of every critical operation performed in the system.